

RAPPORT DE PROJET

Cryptographie : collisions et inversion de MD5 par calcul GPU

Projet intégratif - Sujet 1 - SAE 204

LACOSTE Nathan - BELHARET Adam

Sommaire

1. Introduction
2. Contexte et objectifs du projet
3. Méthodologie et outils utilisés
4. Implémentation : recherche de collisions MD5
5. Implémentation : reverse de MD5
6. Difficultés rencontrées
7. Résultats obtenus
8. Conclusion et perspectives
9. Sources et bibliographie

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du projet intégratif SAE 204, portant sur la cryptographie et plus précisément sur les collisions de l'algorithme de hachage MD5. L'objectif de ce projet était d'étudier les faiblesses connues de MD5, puis de mettre en pratique cette étude en implémentant, en C++ et en utilisant la puissance de calcul parallèle d'une carte graphique (GPU via CUDA), des programmes capables de générer des collisions de hachage et d'inverser des empreintes MD5.

2. Contexte et objectifs du projet

Le MD5 (Message Digest 5) est une fonction de hachage cryptographique permettant d'obtenir l'empreinte numérique d'un fichier ou d'un message. Bien qu'il ait eu un intérêt historique important, cet algorithme est aujourd'hui considéré comme obsolète et impropre à toute utilisation en sécurité informatique, en raison de sa vulnérabilité aux collisions de hachage.

Une collision de hachage survient lorsque deux entrées différentes produisent la même empreinte. Cette faiblesse peut être exploitée par des acteurs malveillants, par exemple pour faire accepter une signature numérique frauduleuse comme légitime.

Les objectifs fixés pour ce projet étaient les suivants :

- Comprendre le fonctionnement de MD5 et les raisons de sa vulnérabilité.
- Implémenter MD5 en C++ exécutable sur GPU (CUDA), aucune bibliothèque MD5 n'existant pour CUDA.
- Mettre en œuvre une recherche de collision par force brute.
- Développer un outil d'inversion de hash MD5 (reverse MD5), par dictionnaire puis par force brute.

3. Méthodologie et outils utilisés

3.1 Choix techniques

Le projet repose sur l'utilisation de CUDA, l'architecture de calcul parallèle de NVIDIA, afin d'exploiter la puissance de traitement d'une carte graphique (GeForce RTX 3060 Ti, 4864 cœurs CUDA). Cette approche est justifiée par la nature du brute force, qui nécessite de tester un très grand nombre de combinaisons : le calcul massivement parallèle d'un GPU, organisé en threads regroupés en warps (32 threads) puis en blocs, est nettement plus performant qu'un traitement séquentiel sur CPU pour ce type de tâche.

Le développement a été réalisé en C++, avec Visual Studio comme environnement de développement, et CUDA Toolkit pour la compilation du code GPU.

3.2 Architecture générale du programme

MD5 ne disposant pas de bibliothèque native pour CUDA, l'algorithme a dû être implémenté manuellement, à partir d'un code de référence trouvé sur Wikipédia. Le programme attribue à chaque thread du GPU le calcul d'un hash. Les résultats sont ensuite stockés et comparés entre eux afin de détecter une éventuelle collision. Une amélioration majeure a consisté à déplacer le stockage de la table de hachage du CPU vers la mémoire de la carte graphique elle-même, ce qui a supprimé un goulot d'étranglement (bottleneck) limitant les performances du programme.

4. Implémentation : recherche de collisions MD5

La recherche de collisions a été menée par force brute, en augmentant progressivement la taille de l'espace de recherche :

- Une première version du programme, basée sur un stockage côté CPU, a permis de trouver une collision sur un espace réduit à 52 bits.
- Après optimisation (stockage de la table de hachage directement en mémoire GPU), une collision sur 64 bits a été trouvée en environ 220 secondes, contre plusieurs minutes sans résultat pour la version initiale.

Sur la base de ce résultat, un calcul théorique a été effectué afin d'estimer le temps nécessaire pour obtenir une collision sur 128 bits. Le débit mesuré est d'environ 19 522 000 hashes par seconde. Une collision nécessitant statistiquement de l'ordre de $2^{(n/2)}$ essais (paradoxe des anniversaires), le temps estimé pour 128 bits est d'environ 944 920 811 070 secondes, soit environ 29 000 ans. Ce résultat illustre concrètement, par le calcul, pourquoi une collision complète sur 128 bits (l'espace réel de MD5 étant de 128 bits) est inatteignable par force brute avec les moyens utilisés, et justifie l'intérêt des méthodes de cryptanalyse mathématique plutôt qu'une approche combinatoire.

5. Implémentation : reverse MD5

5.1 Reverse par dictionnaire

Une première approche, plus simple, a consisté à retrouver le mot de passe correspondant à une empreinte MD5 donnée en la comparant à celles d'une liste de mots de passe (dictionnaire) issue d'une fuite de données de 2009, récupérée sur GitHub. Le test réalisé a permis de retrouver avec succès le mot de passe "hello95" à partir de son hash, en 262 144 tentatives. Cette méthode reste néanmoins limitée par le contenu du dictionnaire utilisé : seuls les mots de passe présents dans la liste peuvent être retrouvés.

5.2 Reverse par force brute

Une seconde approche, plus complexe, visait à inverser un hash MD5 sans dictionnaire préétabli, en testant directement des combinaisons. Le code disponible publiquement pour cette méthode s'est révélé difficile à comprendre, contrairement à l'approche par dictionnaire. Cette piste, ainsi que le test sur un hash artificiel généré pour l'occasion, ont nécessité un travail de recherche complémentaire pour identifier une implémentation suffisamment claire pour être reprise et adaptée.

6. Difficultés rencontrées

La principale difficulté du projet n'a pas été d'ordre algorithmique mais matériel : la mise en place de l'environnement CUDA. La carte graphique n'était initialement pas détectée, ni sur l'ordinateur personnel utilisé, ni sur un poste de la salle informatique. Après de nombreuses recherches infructueuses (pistes liées aux drivers), il s'est avéré que le problème provenait de l'absence de port Thunderbolt sur l'ordinateur personnel et d'un problème de port PCIe sur le poste de la salle. La solution a finalement consisté à utiliser un ordinateur de l'IUT disposant d'un port Thunderbolt compatible, ce qui a permis la reconnaissance correcte de la carte graphique.

Une seconde difficulté a résidé dans la compréhension de codes de reverse MD5 par force brute trouvés en ligne, dont la logique s'est révélée moins accessible que celle du code de hachage MD5 lui-même.

7. Résultats obtenus

Tâche	Résultat
Collision MD5 (CPU, version initiale)	Collision trouvée sur 52 bits
Collision MD5 (GPU optimisé)	Collision trouvée sur 64 bits en = 220 s (= 19,5 M hash/s)
Estimation collision 128 bits	= 29 000 ans par force brute (calcul théorique)
Reverse MD5 par dictionnaire	Mot de passe retrouvé ("hello95") en 262 144 tentatives
Reverse MD5 par force brute	Piste explorée, recherche d'une implémentation compréhensible en cours

8. Conclusion et perspectives

Ce projet a permis de mettre en pratique, de manière concrète, les notions théoriques liées aux collisions de hachage MD5. L'implémentation sur GPU a démontré l'intérêt du calcul parallèle pour les attaques par force brute, tout en révélant ses limites : le calcul théorique du temps nécessaire pour une collision sur 128 bits (environ 29 000 ans) montre que la force brute seule ne permet pas d'exploiter MD5 à grande échelle, et que des méthodes mathématiques plus avancées, telles que celles développées dans la recherche académique sur MD5, sont nécessaires pour des attaques réalistes.

Les perspectives identifiées pour la suite du projet concernent principalement l'inversion de MD5 par force brute, dont l'implémentation reste à finaliser, ainsi que l'extension du reverse par dictionnaire à plusieurs fichiers de mots de passe afin de dépasser la limitation actuelle liée à un dictionnaire unique.

9. Sources et bibliographie

- Stevens, M.M.J. - On Collisions for MD5 (thèse de recherche)
- Wikipédia (FR) - Fonction de hachage cryptographique
- Wikipédia (FR) - Compute Unified Device Architecture (CUDA)
- Okta (FR) - Identity 101 : MD5
- AWS - What is a GPU?
- IBM - Topics : GPU
- Cybertek - Quel processeur pour une RTX 3060 Ti
- CERT/CC - Vulnerability Note VU#836068 : MD5 vulnerable to collision attacks
- Ressources vidéo YouTube consultées sur le hachage, CUDA et les threads GPU (cf. journal de bord pour le détail des liens)